PROGRAM 1 – 8-PUZZLE PROBLEM

Aim

To write a Python program to solve the 8-Puzzle problem using Breadth First Search.

Algorithm

1. Start with the initial 8-puzzle state.
2. Store the state in a queue.
3. Find the position of the blank tile 0.
4. Generate all possible moves of the blank tile.
5. Add unvisited states to the queue.
6. Continue until the goal state is reached.
7. Print the solution path.

Code

```
from collections import deque
def neighbors(state):
    result = []
    zero = state.index(0)
    row, col = divmod(zero, 3)
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    for dr, dc in moves:
        nr, nc = row + dr, col + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            new_pos = nr * 3 + nc
            new_state = list(state)
            new_state[zero], new_state[new_pos] = \
                new_state[new_pos], new_state[zero]
            result.append(tuple(new_state))
    return result
def solve(start, goal):
    queue = deque([(start, [start])])
    visited = {start}
```

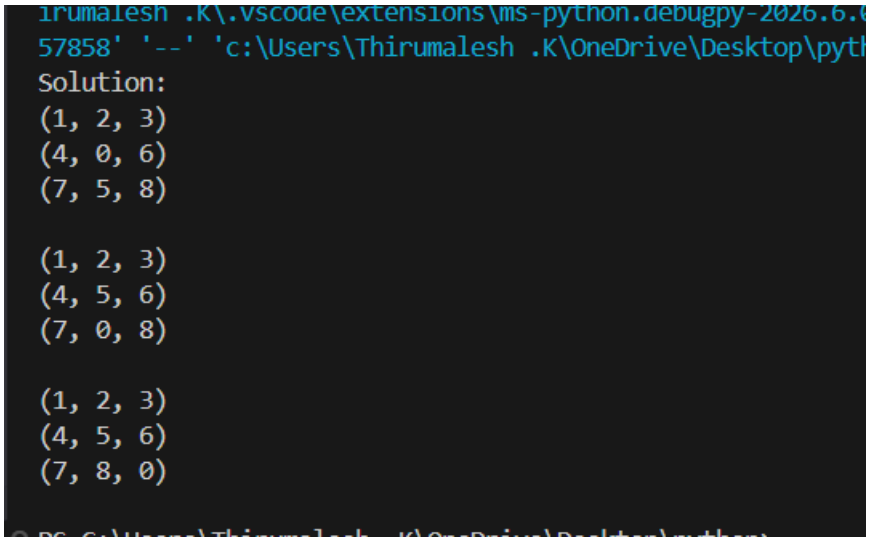
```
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path
        for next_state in neighbors(state):
            if next_state not in visited:
                visited.add(next_state)
                queue.append((next_state, path + [next_state]))
    return None
start = (1, 2, 3,
         4, 0, 6,
         7, 5, 8)
goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)
path = solve(start, goal)
if path:
    print("Solution found:")
    for state in path:
        print(state[0:3])
        print(state[3:6])
        print(state[6:9])
        print()
else:
    print("No solution found")
```

Input

```
start = (1, 2, 3,
         4, 0, 6,
         7, 5, 8)

goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)
```

Output



Result

Thus, the 8-Puzzle problem was successfully solved using Python.